UI/UX за AI продукти

Генеративен интерфейс (Generative UI) и стрийминг на отговорите

Богомил Стоянов, Михаил Доброславски

Лекция 4

Приложно AI програмиране

© 2026 Богомил Стоянов, Михаил Доброславски · CC BY-NC-ND 4.0

От Тема 3 към днес

- Тема 3: моделът връща **надеждни структурирани данни**. Днес: как ги *показваме* на потребителя.
- Проблемът: LLM има висока и **силно променлива** латентност, а класическият подход с въртящ се индикатор (spinner) до готовия резултат разваля UX.
- Решението: **стрийминг** (изходът пристига на части, докато се генерира) и интерфейс, съобразен с вероятностната природа на модела.

Тезата на лекцията

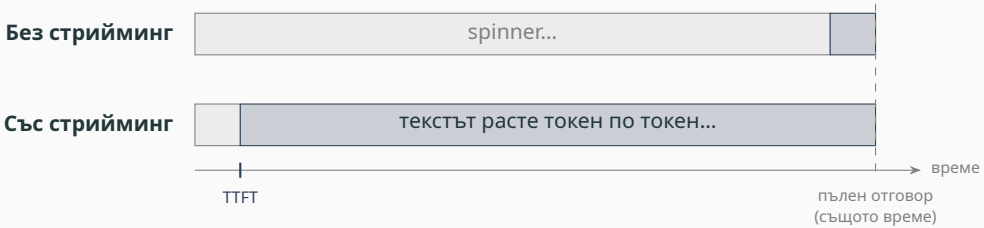
Стриймингът не прави модела по-бърз, а чакането **поносимо**. Не е оптимизация, а *базово изискване* за чат приложенията.

Дневен ред: Лекция 4

- Защо UI/UX за AI е различен
- Механика на стрийминга
- Модел на делтите и собствен endpoint
- Backpressure и рендиране
- Vercel AI SDK и Generative UI
- UX модели за AI интерфейс
- Чести грешки
- Обобщение
- Дискусия

Защо UI/UX за AI е различен

Реална и възприемана латентност



Без стрийминг потребителят гледа spinner 5–30 s и се чуди дали нещо не е „забило“. Със стрийминг текстът расте пред очите още от първата секунда.

Ключово разграничение

Реалната латентност (общото време) е същата. Стриймингът намалява **възприеманата** латентност, т.е. колко бавно се *усеща* системата.

TTFT и вероятностната природа

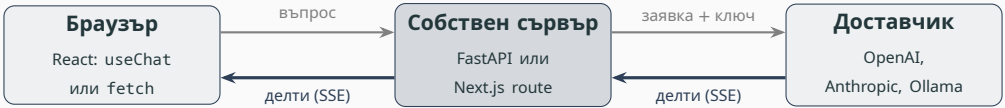
- **Time to First Token (TTFT)**: времето до първия токен. Щом текстът тръгне, системата се усеща „жива“, затова за UX TTFT тежи повече от общото време.
- **Недетерминизъм**: един и същ вход може да даде различен изход. Интерфейсът трябва да го отчита: повторно генериране (*regenerate*), редакция, обратна връзка (палец горе/долу).
- **Несигурност**: моделът може да греши; интерфейсът не бива да представя изхода като абсолютна истина.

Продуктова гледна точка

Оптимизирай **TTFT**, не само общото време. Ориентир от изследванията на използваемостта (Якоб Нилсен): до 1 s мисълта на потребителя не се прекъсва, над 10 s вниманието му се губи.

Механика на стрийминга

Пътят на един токен



- **Защо сървър по средата**: API ключът не стига до браузъра, а сървърът избира модела, добавя системното съобщение и записва изразходваните токени.
- **Делта** е новото парче от отговора (обикновено един или няколко токена). Сървърът я **препредава** веднага, без да чака целия отговор. Делтите пътуват по **SSE** (Server-Sent Events, следващият слайд).
- **Отмяната** върви обратно: браузърът прекъсва връзката → сървърът затваря потока към доставчика → генерацията спира.

Server-Sent Events (SSE)

Server-Sent Events (SSE) е прост стандарт, с който сървърът праща на браузъра поредица от съобщения (*събития*) през една HTTP връзка. Така стриймват OpenAI, Anthropic и Gemini.

```
data: "Здра"

data: "вей,"

data: " как"

data: [DONE]
```

Всяко събитие е ред `data: ...` с една делта, а **празен ред** го завършва. Делтата обикновено е в JSON (тук просто низ в кавички). `data: [DONE]` отбелязва края на потока (при OpenAI).

Сървърът завършва всяко събитие с празен ред

Т.е. след всяко `data: ...` пише `\n\n`. Ако го забрави, браузърът не знае къде свършва едно събитие и започва следващото.

SSE или WebSocket

WebSocket е другият разпространен начин за постоянна връзка: двупосочен канал, по който и браузърът, и сървърът пишат по всяко време.

	SSE	**WebSocket**
Посока	само сървър → браузър	в двете посоки
Сложност	ниска (обикновен HTTP)	по-висока
Подходящ за	**отговор на LLM**	игри, съвместна работа на живо

Отговорът на модела тече само в една посока, затова SSE е достатъчен и по-прост. Въпросът на потребителя пък е отделна, обикновена заявка.

Стрийминг от доставчика в Python (натрупване на делти)

```python
from openai import OpenAI
client = OpenAI()

stream = client.chat.completions.create(
    model="gpt-6-luna",
    messages=[{"role": "user", "content": "Разкажи кратка история."}],
    stream=True,                        # SDK-то скрива SSE зад итератор
)
for chunk in stream:
    delta = chunk.choices[0].delta.content
    if delta:
        print(delta, end="", flush=True)   # рендираме токен по токен
```

Anthropic, със същия модел на делтите:

```python
with anthropic.Anthropic().messages.stream(model="claude-haiku-4-5",
        max_tokens=1024, messages=[...]) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

Същото с локален модел (Ollama)

Стриймингът не е привилегия само на облачните (hosted) API. Работи и локално, без ключ:

```python
import ollama

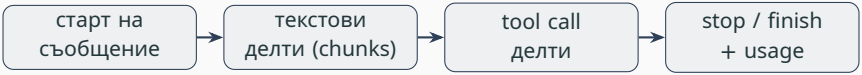
stream = ollama.chat(
    model="bggpt:4b",                  # BgGPT (INSAIT), адаптиран за български
    messages=[{"role": "user", "content": "Разкажи кратка история."}],
    stream=True,
)
for chunk in stream:
    print(chunk["message"]["content"], end="", flush=True)
```

Същият модел на делтите (`stream=True` → итератор от парчета). Още един знак, че тази абстракция е де факто стандарт: облачен и локален модел могат да стоят зад един и същ слой за стрийминг.

Модел на делтите и собствен endpoint

Типове събития в потока (модел на делтите)

Изходът идва като поредица **делти**, не като финални обекти:



- Клиентът **натрупва** делтите в текущо състояние: слепва текста и сглобява JSON аргументите на tool call парче по парче.
- `finish_reason` (OpenAI) / `stop_reason` (Anthropic) показва *защо* генерацията е спряла: `stop` / `length` / `tool_calls` / `content_filter` при OpenAI, `end_turn` / `max_tokens` / `tool_use` / `refusal` при Anthropic. Всеки случай изисква различно поведение на интерфейса.

Собствен SSE endpoint (FastAPI)

```python
def token_stream(prompt: str):
    stream = client.chat.completions.create(model="gpt-6-luna", stream=True,
        messages=[{"role": "user", "content": prompt}])
    for chunk in stream:
        delta = chunk.choices[0].delta.content
        if delta:
            yield f"data: {json.dumps(delta)}\n\n"   # едно SSE събитие
    yield "data: [DONE]\n\n"                         # краят на потока

@app.get("/chat")
def chat(q: str):
    return StreamingResponse(token_stream(q), media_type="text/event-stream")
```

- `token_stream` връща (`yield`) по едно събитие за всяка делта, а `StreamingResponse` го праща на браузъра веднага.
- `json.dumps` записва новите редове в текста като `\n`; суров нов ред би разцепил събитието.

Клиент без библиотека (fetch + четец)

```
const res = await fetch(`/chat?q=${encodeURIComponent(q)}`,
                        { signal: controller.signal });   // за бутона „Спри“
const reader = res.body.getReader();
const decoder = new TextDecoder();
let buffer = "";
while (true) {
  const { value, done } = await reader.read();            // чакаме следващото парче
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  const events = buffer.split("\n\n");
  buffer = events.pop();                                   // незавършеното чака
  for (const evt of events) {
    const data = evt.replace(/^data: /, "");
    if (data === "[DONE]") return;
    output.textContent += JSON.parse(data);                // токен по токен
  }
}
```

Четем отговора парче по парче; капаните са на следващия слайд.

Клиент без библиотека: два капана

1. **Мрежовите парчета не съвпадат със събитията.** Едно четене може да донесе половин събитие или три наведнъж. Затова трупаме входа в `buffer`, режем по празен ред и пазим недовършения край за следващото четене (`events.pop()`).
2. **Една буква може да се разцепи.** В UTF-8 всяка буква на кирилица заема 2 байта и може да попадне на границата между две парчета. `{stream: true}` кара декодера да изчака втория байт; без него на мястото на буквата излиза нечетим символ.

Защо не готовия `EventSource` на браузъра?

Браузърът има вграден SSE клиент, `EventSource`, но той праща само GET заявки и след края на потока сам се свързва отново, т.е. пуска нова генерация. Затова чат приложенията четат потока сами с `fetch`.

Backpressure и рендиране

Backpressure и отмяна

- **Backpressure** (обратно налягане): сървърът може да праща токени по-бързо, отколкото клиентът ги показва. Четенето с `reader.read()` го решава естествено: клиентът „дърпа“ следващото парче, когато е готов.
- **Без междинно буфериране**: прокси сървър (напр. nginx) може да събере целия отговор и чак тогава да го прати, което обезсмисля стрийминга. Сървърът го забранява със заглавката `X-Accel-Buffering: no`.
- **Отмяна**: `AbortController` / отпадане на връзката трябва да *спре* генерацията и на сървъра (той затваря потока към доставчика).

Липса на отмяна

Генерация, която продължава след „Спри“, харчи пари и разваля UX.

Показване на частични токени

- **Наивно**: суровият текст просто се добавя накрая → Markdown и кодът изглеждат „счупени“ до затварящия ```.
- **Практики**:
  - Markdown библиотека, която понася незатворени блокове,
  - курсор / индикатор, че се пише,
  - обновявай екрана през ~30–60 ms (**throttle**), не при всеки токен,
  - автоматичен скрол, който *зачита* ръчния скрол на потребителя,
  - прерисувай само съобщението, което расте, а не целия чат.

Прерисуване на целия чат при всеки токен

При дълги разговори интерфейсът насича и премигва. Решение: throttle + прерисуване само на последното съобщение.

Стрийминг на tool calls

- При function/tool calling (Тема 9) **името** на инструмента идва в началото, а JSON аргументите пристигат *на парчета*. До края те не са валиден JSON.
- **UX**: покажи „Извиквам инструмент X…“ веднага щом дойде името, вместо да чакаш пълните аргументи.
- **Многостъпков поток**: текст → tool call → резултат → пак текст. Затова едно съобщение се пази като списък от *части* (`parts`): парче текст, извикване на инструмент, пак текст.

Парсване на аргументите преди края

`JSON.parse` върху недовършени аргументи хвърля грешка. Аргументите са валиден JSON *едва* когато частта приключи (finish).

Vercel AI SDK и Generative UI

Готовата абстракция: сървър (`streamText`)

Vercel AI SDK: библиотека с отворен код за TypeScript (от създателите на Next.js) с един интерфейс към много доставчици и готов стрийминг за сървъра и React.

```ts
import { openai } from "@ai-sdk/openai";
import { streamText, convertToModelMessages, UIMessage } from "ai";

export async function POST(req: Request) {
  const { messages }: { messages: UIMessage[] } = await req.json();
  const result = streamText({
    model: openai("gpt-6-luna"),
    messages: await convertToModelMessages(messages),
  });
  return result.toUIMessageStreamResponse();   // готов SSE поток към клиента
}
```

Този handler замества `token_stream` + SSE форматирането от по-рано.

Готовата абстракция: клиент (useChat)

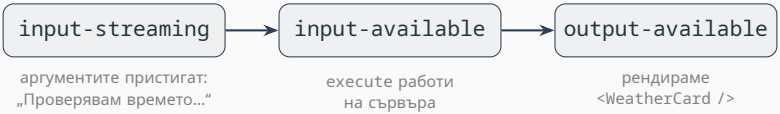
```
"use client";
import { useChat } from "@ai-sdk/react";

function Chat() {
  const { messages, sendMessage, status, stop, error, regenerate } = useChat();
  // messages[i].parts -> рендираме текст / tool-части като компоненти
  // status: "submitted" | "streaming" | "ready" | "error"
  return <>
    {/* ... списък съобщения по parts ... */}
    {status === "streaming" && <button onClick={stop}>Спри</button>}
    {error && <button onClick={() => regenerate()}>Опитай пак</button>}
  </>;
}
```

`useChat` поема оптимистичното добавяне (optimistic append), състоянието на стрийминга, `stop`, грешките и `regenerate`: всичко, което иначе бихме писали ръчно.

Generative UI (генеративен интерфейс)

Идея: моделът не връща само текст, а избира *какъв компонент* да се покаже (карта, таблица, форма, графика). Механизмът е tool calling: всяка tool част на съобщението (извикване на инструмент) минава през три състояния:



Ако инструментът се провали, частта минава в `output-error` и показваме грешка вместо компонента.

Кога кое да изберем

Чист текст стига за прости чатове; Generative UI е за случаите, в които интерактивният изход *реално улеснява* потребителя.

Generative UI: инструментът на сървъра

```ts
import { streamText, tool, convertToModelMessages } from "ai";
import { z } from "zod";

const result = streamText({
  model: openai("gpt-6-luna"),
  messages: await convertToModelMessages(messages),
  tools: {
    weather: tool({                                  // -> част "tool-weather"
      description: "Текущото време за даден град",
      inputSchema: z.object({ city: z.string() }),   // схема, както в Тема 3
      execute: async ({ city }) => getWeather(city), // данни за компонента
    }),
  },
});
return result.toUIMessageStreamResponse();
```

Моделът сам решава дали да извика `weather`; SDK-то изпълнява `execute` на сървъра и праща резултата в същото съобщение.

Generative UI: от части (parts) към компоненти

```
{message.parts.map((part, i) => {
  switch (part.type) {
    case "text":
      return <Markdown key={i}>{part.text}</Markdown>;
    case "tool-weather":                          // tool-<име> част
      if (part.state === "output-available")
        return <WeatherCard key={i} {...part.output} />;   // компонент, не текст
      if (part.state === "output-error")
        return <div key={i}>Грешка: {part.errorText}</div>;
      return <div key={i}>Проверявам времето...</div>;       // докато се стриймва
  }
})}
```

`message.parts` е списъкът от части на съобщението. За всяка гледаме типа: текстът става Markdown, а готовият резултат от инструмента става компонент с типизирани данни (структурираният изход от Тема 3).

Optimistic UI, състояние и грешки

- **Optimistic UI**: потребителското съобщение се показва *веднага*; празното място (placeholder) за отговора на асистента се запълва по време на стрийминга.
- **Състояние на чата** (`useChat`): `messages`, `status`, `sendMessage`, `stop`, `error`, `regenerate`.
- **Грешки/стоп**: мрежов срив по средата на отговора, rate limit, content filter, прекъсване от потребителя. Запази частичния изход и предложи „опитай пак / генерирай отново“.
- **Идемпотентност**: повторен опит (retry) за вече завършена генерация не бива да дублира съобщения, затова следи идентификаторите (`id`).

UX модели за AI интерфейс

UX модели, характерни за AI интерфейса

Контрол за потребителя

- Бутон **„Спри“**, докато се генерира.
- **„Генерирай отново“** и редакция на въпроса.
- **Одобрение от човек** преди действие с последствия (изпращане, плащане, изтриване; Теми 12 и 14).

Прозрачност и доверие

- Видим статус: „Търся…“, „Извиквам инструмент X…“.
- **Източници** към отговора, за да може да се провери (Тема 7).
- Обратна връзка (палец горе/долу) и ясен знак, че AI може да греши.

Празният чат

3–4 примерни въпроса показват какво може продуктът и отговарят на „Какво да го питам?“.

Чести грешки

Чести грешки и важни уточнения

- **Буфериране от прокси сървъра** (напр. nginx) „изяжда“ стрийминга: изключи го.
- **Суров Markdown/код токен по токен** → счупен изглед; ползвай толерантна Markdown библиотека.
- **Прерисуване на целия чат при всеки токен** → насичане; throttle + прерисуване само на последното съобщение.
- **Отмяна, която не стига до доставчика** → прекъснатият чат още харчи токени.
- **Парсване на tool JSON преди края** → грешки; JSON е валиден едва накрая.
- **Игнориране на stop reason**: нормалният край, отрязаният изход (`length` / `max_tokens`) и филтърът или отказът изискват различен UX.
- **SSE събитие без празен ред накрая** или суров нов ред в делтата → събитията се разпадат; кодирай делтата като JSON.
- **Смесване на реалната и възприеманата латентност**: стриймингът прави чакането поносимо, а не модела по-бърз.

Обобщение

Какво научихме днес

- Висока, променлива латентност → оптимизираме **възприеманата** латентност и **TTFT**.
- **SSE** е механизмът: еднопосочен HTTP поток от делти; SDK-тата го скриват зад итератор (`stream=True`). Ollama: същото, но в NDJSON.
- Изградихме SSE **на ниско ниво** (FastAPI + `fetch` и четец), после съзнателно използвахме **Vercel AI SDK** (`streamText` + `useChat`).
- Надежден UX: backpressure, отмяна, throttle, stop reason, optimistic UI, regenerate.
- **Generative UI**: инструмент на сървъра → tool част → React компонент; моделът управлява компоненти, а не само текст.
- AI интерфейсът дава **контрол** (спри, генерирай отново, одобрение) и **прозрачност** (статус, източници, обратна връзка).

Следваща лекция

Тема 5: Embeddings, основата на семантичното търсене и RAG.

Връзка с изпитния конспект

- **Въпрос 13**: „Специфики на потребителския интерфейс при AI приложения (Стрийминг и Generative UI)“ → покрит изцяло:
  - *Стрийминг*: възприемана латентност/TTFT, SSE механика, делти, backpressure, рендиране на частичен изход, стрийминг на tool calls.
  - *Generative UI*: компоненти вместо текст (tool части и техните състояния), optimistic UI, състояние на чата, обработка на грешки и спиране, UX модели за контрол и прозрачност.
- **Връзки**: структурираният изход (Тема 3) захранва Generative UI; стрийминтът на tool делти продължава в Тема 9; агентите (Тема 12) стриймват междинни стъпки.

Дискусия

Теми

1. Стриймингът подобрява ли *реалната* или само *възприеманата* производителност? Кога сложността си струва?
2. Как да проектираме UI, който честно показва несигурността и грешките на модела, без да плаши потребителя?
3. Generative UI: докъде оставяме модела да „сглобява“ интерфейса и къде човек трябва да запази контрол?

Библиография

- Vercel AI SDK: `https://ai-sdk.dev/docs`
- MDN, Server-Sent Events / EventSource: `https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events`
- OpenAI API, Streaming: `https://developers.openai.com/api/docs`
- Anthropic, Streaming Messages: `https://docs.claude.com/`
- Gemini API, стрийминг с `generateContent`: `https://ai.google.dev/gemini-api/docs`
- FastAPI, StreamingResponse: `https://fastapi.tiangolo.com/advanced/custom-response/`
- WHATWG, Streams Standard: `https://streams.spec.whatwg.org/`